Inference in First-Order Logic

What is inference?

Inference means deriving a new statement from statements that are already known.

For example:

Facts Human(Socrates)

Rule

Human(x)  Mortal(x)

Therefore: Mortal(Socrates)

So:

$$Human(Socrates) + Human(x) \Rightarrow Mortal(x)$$

This process of deriving Mortal(Socrates) is called inference.

Method	Basic idea
Propositional inference	Convert FOL statements into specific propositions
Unification	Find substitutions that make two expressions identical
Lifting	Perform inference directly with variables
Forward chaining	Start from facts and derive new facts
Backward chaining	Start from goal and find supporting facts
Resolution	Prove a statement by contradiction

Propositional vs. First-Order Inference

One way to perform FOL inference is to instantiate quantified variables and then use propositional inference.

Propositional logic

In propositional logic, a proposition is a complete statement.

For example: P = John is a student

Q = John studies

We can have: P ⟶ Q

and: P

Therefore: Q

This is ordinary Modus Ponens:

$$\frac{P, \quad P \Rightarrow Q}{Q}$$

Limitation of propositional logic

Suppose we want to express:

All students who study pass the examination.

In propositional logic, we might have to write separately:

$$Student(John) \land Studies(John) \Rightarrow Pass(John)$$

and:

$$Student(Ravi) \land Studies(Ravi) \Rightarrow Pass(Ravi)$$

and:

$$Student(Sita) \land Studies(Sita) \Rightarrow Pass(Sita)$$

This becomes tedious if there are thousands of students.

First-Order Logic solves this using variables

We can simply write:

$$\boxed{\forall x \; Student(x) \land Studies(x) \Rightarrow Pass(x)}$$

Now one rule applies to every object x.

This is one of the major advantages of FOL.

Universal Instantiation

Suppose:

$$\forall x \; Student(x) \land Studies(x) \Rightarrow Pass(x)$$

We can substitute:

$$x = John$$

and obtain:

$$Student(John) \land Studies(John) \Rightarrow Pass(John)$$

This is called **Universal Instantiation (UI)**.

Formally:

$$\boxed{\frac{\forall x \; \alpha}{SUBST(\{x/g\}, \alpha)}}$$

where g is a ground term.

Example

Knowledge base:

$$\forall x \; Student(x) \land Studies(x) \Rightarrow Pass(x)$$

Facts:

$$Student(John)$$

$$Studies(John)$$

Question: Does John pass?

Step 1: Instantiate the rule

$$Student(John) \land Studies(John) \Rightarrow Pass(John)$$

Step 2: Check the premises

We have:

Student(John) and: Studies(John)

Therefore: Pass(John)

Propositional vs FOL

Propositional Logic	First-Order Logic
Uses propositions	Uses predicates
No variables	Has variables
No quantifiers	Has $\forall, \exists$
Limited representation	More expressive
Example: $P \Rightarrow Q$	$Student(x) \Rightarrow Studies(x)$
Treats statements as indivisible	Represents objects and relationships
Easier inference	More computationally complex

Unification and Lifting

This is one of the most important concepts in FOL inference.

The basic problem is: How do we determine whether two FOL expressions can be made identical?

The answer is: Unification

What is unification?

Unification finds a substitution that makes two expressions identical.

Formally: UNIFY(p, q) = θ

such that: SUBST(θ, p) = SUBST(θ, q)

Example

Consider: Knows(John, x)

and: Knows(John, Jane)

We can substitute: x = Jane

Therefore: θ = x / Jane

After substitution: Knows(John, Jane)

on both sides.

Therefore they unify.

Example

Consider: Knows(John,x)

and: Knows(y,Bill)

We need: John = y

and: x = Bill

Therefore: θ = {y/John, x/Bill}

Example with a function

Consider: Knows(John, x)

and: Knows(y, Mother(y))

We need: y = John

Therefore: x = Mother(John)

So: θ = {y/John, x/Mother(John)}

When does unification fail?

Consider: Knows(John, x)

and: Knows(x, Elizabeth)

At first glance, we might think: x = John from the first argument,

but: x = Elizabeth from the second argument.

A single x cannot simultaneously be: John and: Elizabeth

Therefore: UNIFY = FAIL

This apparent problem can be solved by **standardizing variables apart**—renaming one of the variables before unification.

Standardizing apart

Suppose: Knows(x, Elizabeth)

Rename its variable: Knows(x_{17}, Elizabeth)

Now unify: Knows(John, x)

with: Knows(x_{17} , Elizabeth)

We obtain: x_{17} = John

and: x = Elizabeth

Therefore: θ = {x/Elizabeth, x_{17}/John }

This is extremely important in backward chaining and resolution.

Most General Unifier — MGU

There may be more than one possible unifier.

Suppose: Knows(John, x)

and: Knows(y, z)

One possible unifier is: {y/John, x/z}

Another is: {y/John, x/John, z/John}

The first is more general because it places fewer restrictions on the variables.

Therefore it is called the: Most General Unifier (MGU)

What is Lifting?

Ordinary Modus Ponens works with exact propositions: P

and: P $\longrightarrow$ Q

Therefore: Q

But FOL has variables.

For example: Student(x) $\longrightarrow$ Pass(x)

and: Student(John)

We don't need to create a separate rule: Student(John) $\longrightarrow$ Pass(John)

Instead, unification automatically determines: x = John

and derives: Pass(John)

This is called lifting.

Generalized Modus Ponens as a lifted version of Modus Ponens, where unification supplies the required substitutions.

Generalized Modus Ponens

Suppose:

$$Student(x) \wedge Studies(x) \Rightarrow Pass(x)$$

Facts: Student(John)

Studies(John)

Unification gives: θ = {x/John}

Apply the substitution: Pass(x)

becomes: Pass(John)

Therefore: Pass(John)

Forward Chaining

Now we come to the first major inference algorithm.

Start from known facts, apply rules whose premises are satisfied, add their conclusions, and continue until the query is answered or no new facts can be generated.

So:

$$\boxed{Facts \rightarrow Rules \rightarrow New\ Facts \rightarrow Rules \rightarrow \ldots}$$

It is therefore called:

$$\boxed{\text{Data-driven inference}}$$

Simple example

Knowledge base:

Facts Student(John)

Studies(John)

Rule 1

$$Student(x) \land Studies(x) \Rightarrow Pass(x)$$

Rule 2

$$Pass(x) \Rightarrow Graduate(x)$$

Question: Is John a graduate?

Forward chaining — Step 1

Initial facts: Student(John)

Studies(John)

Rule 1 matches.

Therefore: Pass(John)

Step 2

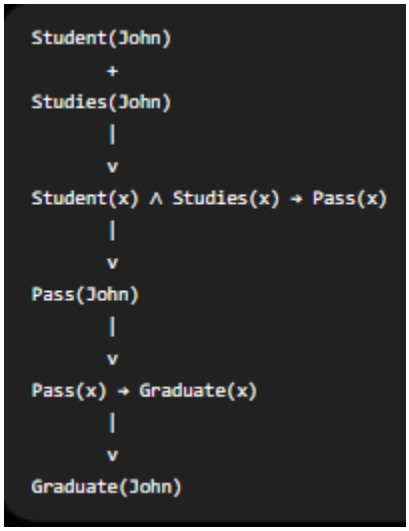
Now we have: Pass(John)

Rule 2 matches.

Therefore: Graduate(John)

So: Graduate(John)

Forward-chaining diagram



West–Nono example

Facts include: American(West)

Enemy(Nono, America)

and: Missile(M1)

Owns(Nono,M1)

Rules include:

$$Missile(x) \land Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$$

$$Missile(x) \Rightarrow Weapon(x)$$

$$Enemy(x, America) \Rightarrow Hostile(x)$$

and:

$$American(x) \land Weapon(y) \land Sells(x, y, z) \land Hostile(z) \Rightarrow Criminal(x)$$

..

$$Sells(West, M1, Nono)$$

$$Weapon(M1)$$

$$Hostile(Nono)$$

Then in the second iteration it derives:

$$\boxed{Criminal(West)}$$

Advantages of forward chaining

Advantage 1 — Data driven

It starts from available facts.

Advantage 2 — Can derive many conclusions

It can continue until a fixed point is reached.

Advantage 3 — Useful for databases

its use in deductive databases and production systems.

Disadvantage

Suppose the knowledge base has: 1000 rules

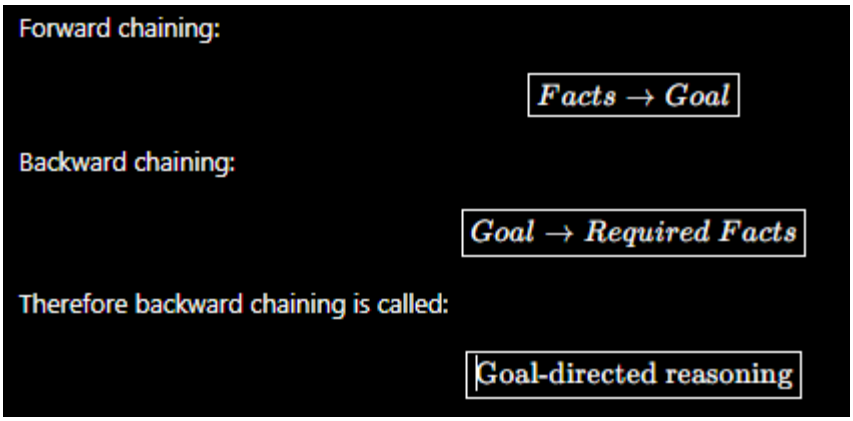
but your question concerns only: Is John a graduate?

Forward chaining may derive many facts unrelated to John being a graduate.

This generation of irrelevant conclusions as one of the main sources of inefficiency.

That leads naturally to: Backward Chaining

Backward Chaining



Simple example

Knowledge base:

Student(John)

Studies(John)

Rule:

$$Student(x) \land Studies(x) \Rightarrow Pass(x)$$

Query:

$$Pass(John)?$$

Step 1

Start with the goal:

$$Pass(John)$$

Ask:

> Which rule can conclude `Pass(x)`?

We find:

$$Student(x) \land Studies(x) \Rightarrow Pass(x)$$

Step 2

Unify:

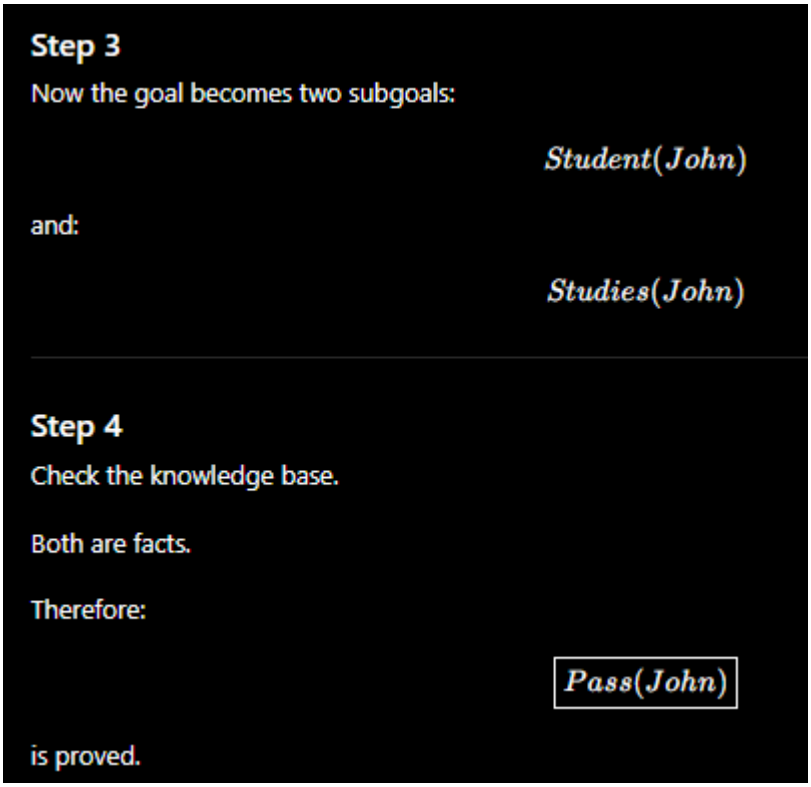
$$Pass(x)$$

with:

$$Pass(John)$$

Therefore:

$$\boxed{x = John}$$



Forward vs backward

Suppose:

Facts:

Student(John)

Studies(John)

Teacher(Ravi)

Doctor(Sita)

...

and the question is:

Pass(John)?

Forward chaining

Starts with:

Student(John)

Studies(John)

Teacher(Ravi)

Doctor(Sita)

...

and tries to derive everything possible.

Backward chaining

Starts directly with:

Pass(John)?

and asks:

What rule can produce Pass?

This is why backward chaining can avoid irrelevant inference.

AND/OR interpretation

describes backward chaining as an AND/OR search.

OR

There may be several rules that can prove the same goal.

For: Pass(John)

perhaps:

$$Student(x) \land Studies(x) \Rightarrow Pass(x)$$

or:

$$TopRank(x) \Rightarrow Pass(x)$$

So we have an **OR** choice.

AND

If we choose:

$$Student(x) \land Studies(x) \Rightarrow Pass(x)$$

then **both** conditions must be satisfied.

Therefore:

$$Student(John)$$

AND:

$$Studies(John)$$

must be true.

Advantages of backward chaining

- Goal-directed
- Avoids many irrelevant facts
- Good for answering specific queries
- Foundation of logic programming
- Used by systems such as Prolog

backward chaining is used in logic programming systems and that Prolog uses depth-first backward chaining.

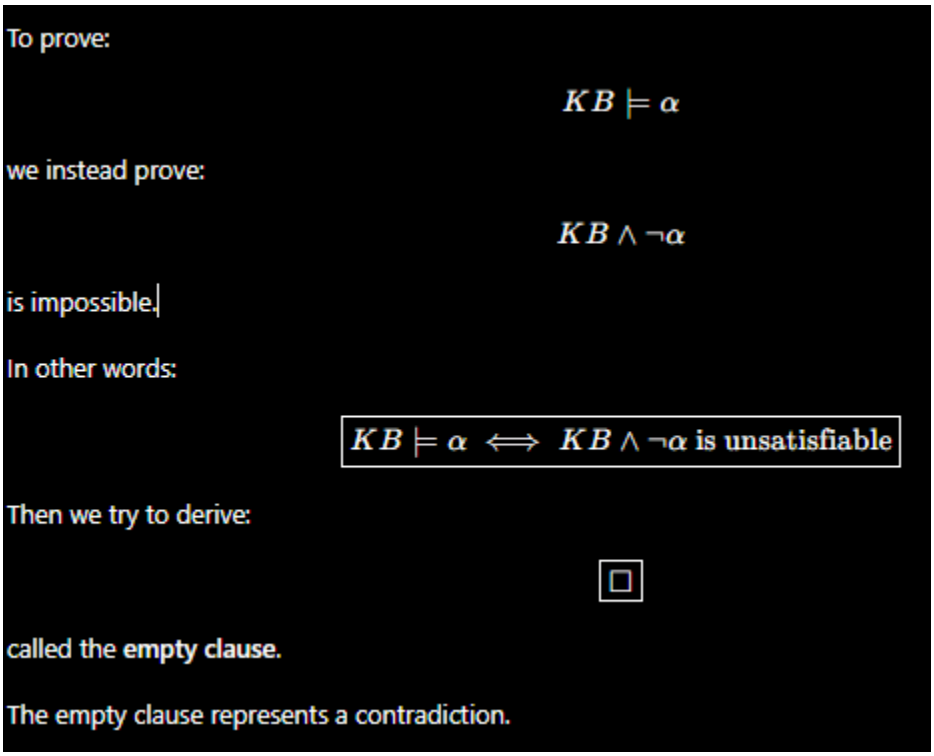
Disadvantages

Backward chaining can suffer from:

- repeated states
- redundant computations
- infinite loops

Resolution

Resolution uses **proof by contradiction**.



Simple propositional resolution

Suppose:

$$P \vee Q$$

and:

$$\neg Q$$

We can resolve them.

The complementary literals are:

$$Q$$

and:

$$\neg Q$$

Remove them and retain the remaining literals:

$$\boxed{P}$$

So:

$$\frac{P \vee Q, \quad \neg Q}{P}$$

FOL resolution

In FOL, the complementary literals do not necessarily have to look exactly identical.

They can unify.

The rule:

If two clauses contain complementary literals that can be unified using substitution θ remove those literals and combine the remaining literals after applying θ.

Simple FOL resolution example

Consider:

$$C_1 : Animal(F(x)) \vee Loves(G(x), x)$$

and:

$$C_2 : \neg Loves(u, v) \vee \neg Kills(u, v)$$

We want to resolve:

$$Loves(G(x), x)$$

with:

$$\neg Loves(u, v)$$

Step 1: Unify

We need:

$$Loves(G(x), x) = Loves(u, v)$$

Therefore:

$$u = G(x)$$

and:

$$v = x$$

Thus:

$$\boxed{\theta = \{u/G(x), v/x\}}$$

Step 2: Remove complementary literals

From:

$$Animal(F(x)) \vee Loves(G(x), x)$$

remove:

$$Loves(G(x), x)$$

From:

$$\neg Loves(u, v) \vee \neg Kills(u, v)$$

remove:

$$\neg Loves(u, v)$$

Step 3: Apply substitution

We obtain:

$$\boxed{Animal(F(x)) \vee \neg Kills(G(x), x)}$$

This is the **resolvent**.

Why do we need CNF?

Before resolution is applied, FOL sentences are converted into: Conjunctive Normal Form (CNF)

CNF is:

$$Clause_1 \wedge Clause_2 \wedge \cdots$$

where each clause is an OR of literals.

For example:

$$(A \vee B) \wedge (\neg C \vee D)$$

Small resolution example

Let's prove:

$$Mortal(Socrates)$$

Given:

$$Human(Socrates)$$

and:

$$\forall x \; Human(x) \Rightarrow Mortal(x)$$

Step 1: Negate the query

Original query:

$$Mortal(Socrates)$$

Negation:

$$\neg Mortal(Socrates)$$

Step 2: Convert the rule

$$Human(x) \Rightarrow Mortal(x)$$

is equivalent to:

$$\neg Human(x) \vee Mortal(x)$$

So our clauses are:

$$C_1 : Human(Socrates)$$

$$C_2 : \neg Human(x) \vee Mortal(x)$$

$$C_3 : \neg Mortal(Socrates)$$

Step 3: Resolve C_1 and C_2

Unify:

$$Human(x)$$

with:

$$Human(Socrates)$$

giving:

$$\theta = \{x/Socrates\}$$

Therefore:

$$Mortal(Socrates)$$

Step 4: Resolve with the negated query

We now have:

$$Mortal(Socrates)$$

and:

$$\neg Mortal(Socrates)$$

Resolution gives:

$$\square$$

Therefore contradiction has been obtained.

Hence:

$$\boxed{Mortal(Socrates)}$$

is proved.

One Example Comparing Forward, Backward and Resolution

Let's use the same knowledge base throughout.

Facts

$$Student(John)$$

$$Studies(John)$$

Rule

$$Student(x) \land Studies(x) \Rightarrow Pass(x)$$

Query

$$Pass(John)?$$

Forward chaining

Start from facts:

$$Student(John)$$

$$Studies(John)$$

Apply rule:

$$Student(x) \land Studies(x) \Rightarrow Pass(x)$$

Unification:

$$\{x/John\}$$

Conclusion:

$$\boxed{Pass(John)}$$

Direction

$$\boxed{Facts \rightarrow Goal}$$

Backward chaining on the same example

Start with:

$$Pass(John)$$

Find rule that concludes `Pass`.

$$Student(x) \land Studies(x) \Rightarrow Pass(x)$$

Unify:

$$x = John$$

Subgoals:

$$Student(John)$$

and:

$$Studies(John)$$

Both are facts.

Therefore:

$$\boxed{Pass(John)}$$

Direction

$$\boxed{Goal \rightarrow Facts}$$

Resolution on the same example

Convert rule:

$$Student(x) \land Studies(x) \Rightarrow Pass(x)$$

to:

$$\neg Student(x) \lor \neg Studies(x) \lor Pass(x)$$

Negate query:

$$\neg Pass(John)$$

Facts:

$$Student(John)$$

$$Studies(John)$$

Now resolve:

$$Student(John)$$

with:

$$\neg Student(x) \lor \neg Studies(x) \lor Pass(x)$$

giving:

$$\neg Studies(John) \vee Pass(John)$$

Resolve with:

$$Studies(John)$$

giving:

$$Pass(John)$$

Finally:

$$Pass(John)$$

and:

$$\neg Pass(John)$$

give:

$$\square$$

Therefore:

$$\boxed{Pass(John)}$$

Topic	Starts with	Main operation	Direction
Propositional inference	Ground propositions	Logical inference	Depends on method
Unification	Two FOL expressions	Find substitution	Matching
Lifting	FOL rules + facts	Unification + generalized MP	Generalized
Forward chaining	Known facts	Apply rules	**Forward**
Backward chaining	Query/goal	Find supporting rules	**Backward**
Resolution	KB + negated query	Resolve complementary literals	Refutation

Feature	Forward Chaining	Backward Chaining
Also called	Data-driven	Goal-driven
Starting point	Facts	Goal
Direction	Facts → conclusion	Goal → premises
Main question	What can I derive?	What do I need to prove?
Irrelevant inference	More likely	Less likely
Useful for	Databases, production systems	Query answering, logic programming
Search style	Data-driven	AND/OR search
Prolog	No	Yes
Can have infinite loops	Generally not in finite Datalog closure	Yes, with unsuitable rule ordering

Why Unification is so Important

Unification is not a separate isolated inference algorithm.

It is the matching mechanism used by the other FOL inference methods.

For example:

Forward chaining

Rule:

Student(x) → Pass(x)

Fact:

Student(John)

Unification:

x = John

Therefore:

Pass(John)

Backward chaining

Goal:

Pass(John)

Rule:

Student(x) → Pass(x)

Unification:

x = John

New goal:

Student(John)

Resolution

P(x) ∨ Q(x)

¬P(John)

Unification:

x = John

Therefore:

Q(John)

So:

Unification is the bridge that allows FOL inference to work with variables.

Why is Lifting important?

Without lifting, we could first convert:

$$Student(x) \Rightarrow Pass(x)$$

into:

$$Student(John) \Rightarrow Pass(John)$$

then:

$$Student(Ravi) \Rightarrow Pass(Ravi)$$

then:

$$Student(Sita) \Rightarrow Pass(Sita)$$

and so on.

This is **propositionalization.**

With lifting, we retain:

$$Student(x) \Rightarrow Pass(x)$$

and use:

$$\{x/John\}$$

only when needed.

Therefore: Lifting avoids unnecessary grounding of all possible objects.

Example:

using a college student knowledge base.

Knowledge base

Facts:

Student(Anu)

Studies(Anu)

Student(Ravi)

Studies(Ravi)

Rules:

$$Student(x) \land Studies(x) \Rightarrow Pass(x)$$

$$Pass(x) \Rightarrow Graduate(x)$$

Propositional inference

Ground:

$$Student(Anu) \land Studies(Anu) \Rightarrow Pass(Anu)$$

Unification

Match:

$$Student(x)$$

with:

$$Student(Anu)$$

giving:

$$\boxed{\{x/Anu\}}$$

Lifting

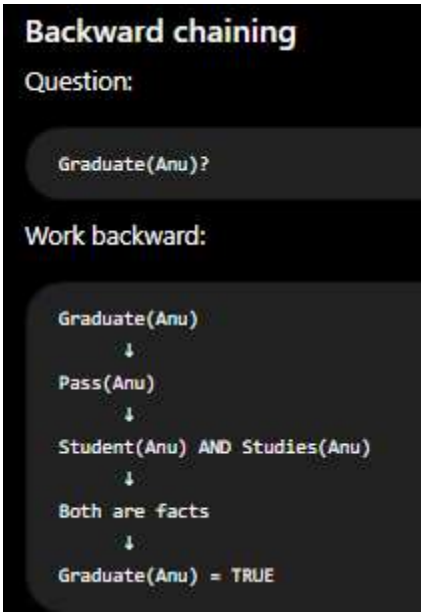
Instead of creating separate rules for Anu and Ravi:

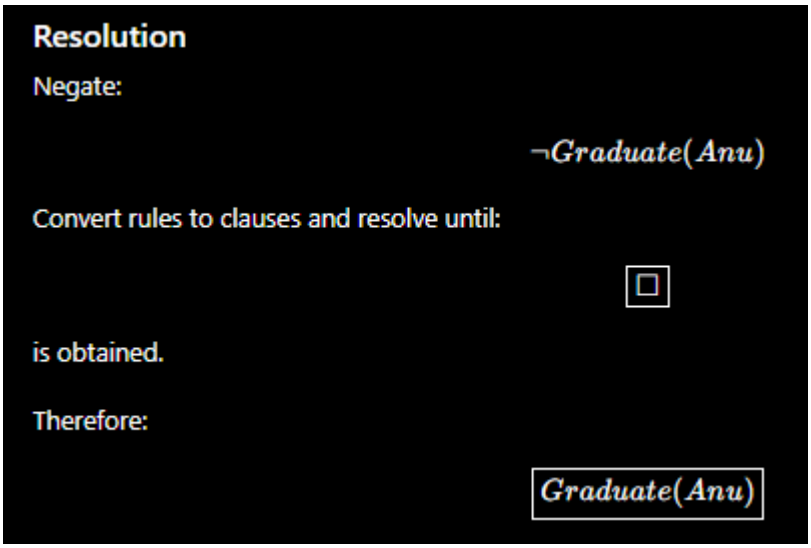
$$Student(x) \land Studies(x) \Rightarrow Pass(x)$$

works for both.

Forward chaining

```
Student(Anu)
Studies(Anu)
      ↓
Pass(Anu)
      ↓
Graduate(Anu)
```





Propositional vs FOL inference

Propositional inference works with ground propositions, whereas FOL inference represents objects, relations, variables and quantifiers. FOL can be propositionalized through instantiation, but this may be computationally expensive.

Unification

Unification finds a substitution that makes two logical expressions identical.

UNIFY(p, q) = θ

Lifting

Lifting extends propositional inference to FOL by using variables and unification rather than explicitly generating all ground instances.

Forward chaining

Forward chaining is data-driven: start from known facts, apply applicable rules, and continue until the query is derived or no new facts can be generated.

$$Facts \rightarrow Rules \rightarrow New\ Facts$$

Backward chaining

Backward chaining is goal-driven: start with the query and recursively find rules and facts that can establish the query.

$$Goal \rightarrow Rules \rightarrow Subgoals \rightarrow Facts$$

Resolution

Resolution proves a query by adding its negation to the knowledge base and deriving the empty clause through resolution.

$$KB \wedge \neg Query \Rightarrow \square$$

this refutation completeness: if the set of sentences is unsatisfiable, resolution can derive a contradiction.

Reasoning under Uncertainty

Reasoning under Uncertainty: Need for uncertainty handling in AI, Conditional probability, Bayes' theorem, Bayesian networks, and Naive Bayes classifier

Need for Uncertainty Handling in AI

What is uncertainty?

In classical logic, a statement is generally either: True False

For example: "Ravi is a student."

We may represent: Student(Ravi) = True

But real-world AI systems frequently do not have complete or perfectly reliable information.

For example: "Will it rain tomorrow?"

We cannot normally say with certainty: RainTomorrow = True

Or RainTomorrow = False

Instead, we may say: P(RainTomorrow) = 0.8

meaning the agent believes there is an 80% probability of rain.

An agent's inability to reach a definite decision regarding the truth of a proposition; probabilities summarize the agent's beliefs relative to the available evidence.

Why does AI need uncertainty handling?

Identifies uncertainty as arising because of factors such as ignorance, partial observability, complexity and nondeterminism.

Consider an autonomous car.

It may observe:

- Camera image → possibly a pedestrian
- Radar → object detected
- GPS → approximate position
- Weather → uncertain

- Road condition → uncertain

The car has to make a decision even though it does not know everything.

For example:

Camera detects object
↓
Is it a pedestrian?
↓
Maybe 90% pedestrian
↓
Should the car brake?
↓
Decision

A purely logical system may have difficulty if the evidence is incomplete.

A probabilistic AI system can reason: P(Pedestrian|Evidence) = 0.90

and use that probability when selecting an action.

Example: Medical diagnosis

Suppose a patient has:
- Fever
- Headache
- Stiff neck

The AI system wants to determine: Does the patient have meningitis?

The evidence does not necessarily prove the disease.

Instead: P(Meningitis|Symptoms)

can be calculated.

A particularly useful example involving meningitis and stiff neck. It assumes:

P(StiffNeck|Meningitis) = 0.7
P(Meningitis)= 1/50000}
P(StiffNeck) = 0.01

Using Bayes' theorem, obtains: P(Meningitis|StiffNeck) = 0.0014

So even though meningitis produces a stiff neck 70% of the time, a person with a stiff neck still has only a very small probability of having meningitis because meningitis itself is extremely rare.

A strong symptom does not necessarily mean a high probability of disease.

The prior probability matters.

Conditional Probability

Conditional probability is one of the most important concepts because it allows AI to reason using evidence.

Suppose:

A = Student passes the exam
B = Student studied

We may want to know: What is the probability that the student passes given that the student studied?

We write: P(A|B)

and read it as: Probability of A given B.

Use conditional probabilities extensively as the foundation for probabilistic reasoning and Bayesian networks.

Why conditional probability is important in AI

Suppose an AI system observes:
Evidence:
Fever = Yes
Cough = Yes

It may want to calculate: P(Disease|Fever,Cough)

The evidence changes the probability of the hypothesis.

This idea is fundamental to:

- Medical diagnosis
- Spam detection
- Fault diagnosis

- Speech recognition
- Recommendation systems
- Fraud detection
- Intrusion detection

Bayes' Theorem

Bayes' theorem is essentially a way of reversing conditional probability.

Why do we need Bayes' theorem?
Suppose we know:

If a person has a disease, what is the probability that they show a symptom?

That is: P(Symptom|Disease)

But in diagnosis, we actually want:

If the person has the symptom, what is the probability that they have the disease?

That is: P(Disease|Symptom)

These are not the same thing.

Bayes' theorem allows us to move from:

P(Symptom|Disease)

to:

P(Disease|Symptom)

Derive Bayes' rule from the product rule and identifies it as a fundamental equation for probabilistic inference.

Derivation

From the product rule:

$$P(A \cap B) = P(A|B)P(B)$$

Also:

$$P(A \cap B) = P(B|A)P(A)$$

Therefore:

$$P(A|B)P(B) = P(B|A)P(A)$$

Rearranging:

$$\boxed{P(A|B) = \frac{P(B|A)P(A)}{P(B)}}$$

This is **Bayes' theorem.**

Bayes' theorem:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Posterior = Likelihood × Prior / Evidence

Let:

- M = patient has meningitis
- S = patient has stiff neck

Given:

$$P(S|M) = 0.7$$

$$P(M) = \frac{1}{50000}$$

$$P(S) = 0.01$$

We want:

$$P(M|S)$$

Using Bayes:

$$P(M|S) = \frac{P(S|M)P(M)}{P(S)}$$

Substitute:

$$P(M|S) = \frac{0.7 \times \frac{1}{50000}}{0.01}$$

$$= \boxed{0.0014}$$

A simple AI example

Suppose an email system wants to determine whether an email is spam.

Let:

S = Spam
W = email contains the word "offer"

Suppose: P(S) = 0.20
P(W|S) = 0.80
P(W) = 0.40

We want: P(S|W)

Using Bayes:

$$P(S|W) = \frac{P(W|S)P(S)}{P(W)}$$

$$= \frac{0.80 \times 0.20}{0.40}$$

$$= \boxed{0.40}$$

Therefore:

$$P(Spam|Offer) = 40\%$$

Bayesian Networks

Now we move from individual probability calculations to a network representation of uncertain knowledge.

Bayesian networks as a systematic way of representing dependencies and conditional-independence relationships.

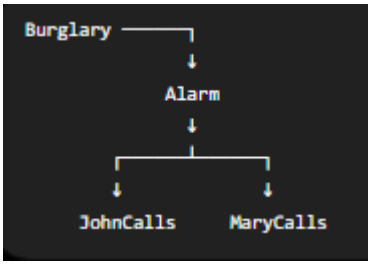
What is a Bayesian Network?

A Bayesian network is a: Directed Acyclic Graph (DAG) in which nodes represent random variables and links represent dependencies between variables, with conditional probability information associated with the nodes.

The textbook specifies that:

- Each node represents a random variable.
- Directed links connect variables.
- The graph contains no directed cycles.
- Each variable has a conditional probability distribution given its parents.

Simple example
Consider:



This is a simplified version of the classic burglary network.

The meaning is:

Burglary → Alarm
Earthquake → Alarm
Alarm → JohnCalls
Alarm → MaryCalls

The idea is:

- Burglary can cause the alarm.
- Earthquake can cause the alarm.
- If the alarm sounds, John may call.
- If the alarm sounds, Mary may call.

This network to demonstrate probabilistic inference.

Why not simply use a huge probability table?

Suppose we have 10 Boolean variables.

Each variable has two possible values: True, False

Therefore the number of possible combinations is:

$$2^{10} = 1024$$

For 20 variables:

$$2^{20} = 1,048,576$$

For 100 variables:

$$2^{100}$$

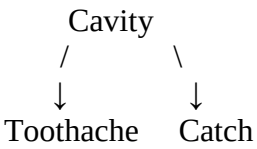
This becomes enormous.

A joint probability distribution can become intractably large as the number of variables increases. Bayesian networks provide a much more concise representation by exploiting independence and conditional independence.

Conditional independence

This is the key idea behind Bayesian networks.

Suppose:



If we know whether there is a cavity, Toothache and Catch can be treated as conditionally independent.

P(Toothache,Catch,Cavity)

as:

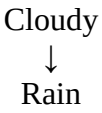
P(Toothache|Cavity) P(Catch|Cavity) P(Cavity)

instead of representing one large table.

Conditional Probability Table — CPT

Each Bayesian network node has probability information.

For example:



Suppose:

$$P(Rain|Cloudy) = 0.8$$

$$P(Rain|\neg Cloudy) = 0.2$$

We can write:

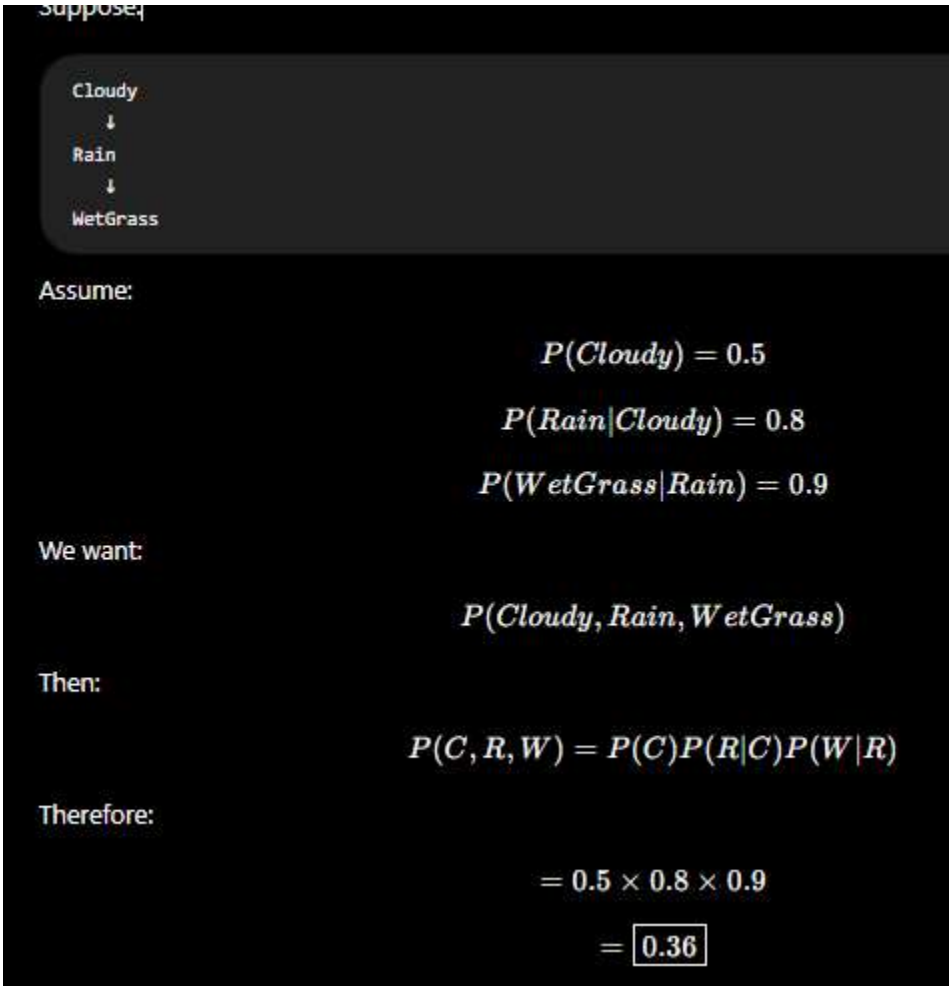
Cloudy	P(Rain)
True	0.8
False	0.2

This is a Conditional Probability Table (CPT).

Joint probability in a Bayesian network

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i | Parents(X_i))$$

The probability of a complete assignment is obtained by multiplying the appropriate conditional probabilities from the network.



Naive Bayes Classifier

Now we come to the machine-learning application of the above concepts.

Naive Bayes uses Bayes' theorem together with a strong simplifying assumption:

Conditional probability

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Product rule

$$P(A \cap B) = P(A|B)P(B)$$

Bayes' theorem

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Bayesian network joint distribution

$$P(x_1, \ldots, x_n) = \prod_i P(x_i|Parents(X_i))$$

Naive Bayes

$$P(C|F_1, \ldots, F_n) \propto P(C) \prod_i P(F_i|C)$$

Topic	Student should remember
Need for uncertainty	Real world is incomplete and uncertain
Conditional probability	Probability given evidence
Bayes' theorem	Reverse (P(Evidence
Bayesian network	Graphical representation of probabilistic dependencies
Naive Bayes	Bayes theorem + conditional independence assumption for classification